

Namespace OceanApocalypseStudios. RSML

Interfaces

[IToolchainComponent](#)

A component of the RSML toolchain.

Enums

[ToolchainConfiguration](#)

Configuration options for a [IToolchainComponent](#).

Interface IToolchainComponent

Namespace: [OceanApocalypseStudios.RSML](#)

Assembly: RSML.dll

A component of the RSML toolchain.

```
public interface IToolchainComponent : IDisposable
```

Inherited Members

[IDisposable.Dispose\(\)](#)

Properties

[Configuration](#)

Configurations for the toolchain component.

Methods

[Inject\(ToolchainConfiguration\)](#)

Injects a configuration into the toolchain component, modifying it.

See Also

[Lexer](#)

[Parser](#)

[Interpreter](#)

Property Configuration

Namespace: [OceanApocalypseStudios.RSML](#)

Assembly: RSML.dll

Configuration

Configurations for the toolchain component.

```
ToolchainConfiguration Configuration { get; }
```

Property Value

[ToolchainConfiguration](#)

Method Inject

Namespace: [OceanApocalypseStudios.RSML](#)

Assembly: RSML.dll

Inject(ToolchainConfiguration)

Injects a configuration into the toolchain component, modifying it.

```
void Inject(ToolchainConfiguration configuration)
```

Parameters

configuration [ToolchainConfiguration](#)

The configuration to inject.

Enum ToolchainConfiguration

Namespace: [OceanApocalypseStudios.RSML](#)

Assembly: RSML.dll

Configuration options for a [IToolchainComponent](#).

[Flags]

```
public enum ToolchainConfiguration
```

Fields

DisableExtensionProcessing = 1

Optimizes the toolchain pipeline by disabling extension processing.

⚠ **WARNING**

This completely disables extensions, but does not warn you if there are active extensions, meaning sometimes you might be wondering why your extension is not working when, in reality, you've enabled this configuration.

i TIP

This configuration is automatically enabled when no extensions are enabled.

AllowOnlyOASExtensions = 2

Only allows OceanApocalypseStudios extensions, leading to an error if any non-OAS extension is active.

i NOTE

When used alongside [IgnoreAllExtensibilityErrors](#), the non-OAS extensions will be disabled, but no errors will be thrown.

IgnoreBrokenExtensions = 4

Ignores all errors caused by broken or faulty extensions.

`IgnoreDuplicatedExtensions = 8`

Ignores all errors caused by injecting already injected extensions.

`IgnoreAllExtensibilityErrors = IgnoreBrokenExtensions |
IgnoreDuplicatedExtensions`

Ignores all errors thrown during pipeline creation and pipeline execution.

⊗ **CAUTION**

This option is only needed in beyond extremely rare occasions. It emulates RSML v1.x.x behavior.

Namespace OceanApocalypseStudios. RSMML.Cache

Interfaces

[ISupportsCache](#)

Represents a service or a type that supports cached data.

Interface ISupportsCache

Namespace: [OceanApocalypseStudios.RSML.Cache](#)

Assembly: RSML.dll

Represents a service or a type that supports cached data.

```
public interface ISupportsCache
```

Properties

[CacheExists](#)

Whether there's cached data.

Methods

[BuildCache\(\)](#)

Builds the cache if it doesn't exist yet.

[BuildCache\(bool\)](#)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

Property CacheExists

Namespace: [OceanApocalypseStudios.RSML.Cache](#)

Assembly: RSML.dll

CacheExists

Whether there's cached data.

```
bool CacheExists { get; }
```

Property Value

[bool](#) 

Method BuildCache

Namespace: [OceanApocalypseStudios.RSML.Cache](#)

Assembly: RSML.dll

BuildCache()

Builds the cache if it doesn't exist yet.

```
void BuildCache()
```

BuildCache(bool)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

```
void BuildCache(bool forceRebuild)
```

Parameters

`forceRebuild` [bool](#)

Whether to force the cache to be built even if it exists.

Namespace OceanApocalypseStudios. RSMML.Diagnostics

Classes

[DiagnosticCollector](#)

A list of RSMML toolchain errors.

Structs

[Diagnostic](#)

A diagnostic reported by RSMML's API.

[ErrorCode](#)

Represents a code specific to an error.

[Result<TValue>](#)

An operation's result.

Enums

[ErrorCategory](#)

The category of an error, which changes its error code.

[Severity](#)

The severity of an error.

Struct Diagnostic

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

A diagnostic reported by RSML's API.

```
public readonly struct Diagnostic : IFormattable, IEquatable<Diagnostic>
```

Implements

[IFormattable](#)[↗], [IEquatable](#)[↗] <[Diagnostic](#)>

Inherited Members

[object.Equals\(object, object\)](#)[↗], [object.GetType\(\)](#)[↗],
[object.ReferenceEquals\(object, object\)](#)[↗]

Constructors

[Diagnostic\(ErrorCode, SourceSpan, string, Severity\)](#)

A diagnostic reported by RSML's API.

Properties

[Category](#)

The error's category.

[Code](#)

The error's code. Contains information about the category of the error.

[IsInternal](#)

Checks whether the error is internal (API error results, for example).

[Message](#)

A brief error message detailing why it has happened.

[Severity](#)

The error's severity.

[Span](#)

The span the error relates to.

Methods

[Equals\(Diagnostic\)](#)

Indicates whether the current object is equal to another object of the same type.

[Equals\(object?\)](#)

Indicates whether this instance and a specified object are equal.

[GetHashCode\(\)](#)

Returns the hash code for this instance.

[ToString\(\)](#)

Returns a generic string representation of the current instance.

[ToString\(string?, IFormatProvider?\)](#)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

Operators

[operator ==\(Diagnostic, Diagnostic\)](#)

Checks if two [Diagnostics](#) are equal to each other.

[operator !=\(Diagnostic, Diagnostic\)](#)

Checks if two [Diagnostics](#) are different from each other.

Constructor Diagnostic

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Diagnostic(ErrorCode, SourceSpan, string, Severity)

A diagnostic reported by RSML's API.

```
public Diagnostic(ErrorCode code, SourceSpan span, string message,
    Severity severity)
```

Parameters

code [ErrorCode](#)

The error code.

span [SourceSpan](#)

The span the error relates to.

message [string](#) 

A brief error message detailing why it has happened.

severity [Severity](#)

The error's severity.

Property Category

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Category

The error's category.

```
public ErrorCategory Category { get; }
```

Property Value

[ErrorCategory](#).

Property Code

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Code

The error's code. Contains information about the category of the error.

```
public ErrorCode Code { get; }
```

Property Value

[ErrorCode](#)

Property IsInternal

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

IsInternal

Checks whether the error is internal (API error results, for example).

```
public bool IsInternal { get; }
```

Property Value

[bool](#)

Property Message

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Message

A brief error message detailing why it has happened.

```
public string Message { get; }
```

Property Value

[string](#)

Property Severity

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Severity

The error's severity.

```
public Severity Severity { get; }
```

Property Value

[Severity](#)

Property Span

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Span

The span the error relates to.

```
public SourceSpan Span { get; }
```

Property Value

[SourceSpan](#)

Method Equals

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current instance.

Returns

[bool](#)

[true](#) if **obj** and this instance are the same type and represent the same value; otherwise, [false](#).

Equals(Diagnostic)

Indicates whether the current object is equal to another object of the same type.

```
public bool Equals(Diagnostic other)
```

Parameters

other [Diagnostic](#)

An object to compare with this object.

Returns

[bool](#)

[true](#) if the current object is equal to the `other` parameter; otherwise, [false](#).

Method GetHashCode

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

[int](#)

A 32-bit signed integer that is the hash code for this instance.

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

ToString()

Returns a generic string representation of the current instance.

```
public override string ToString()
```

Returns

[string](#) 

The string representation.

ToString(string?, IFormatProvider?)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

```
public string ToString(string? format, IFormatProvider? formatProvider)
```

Parameters

format [string](#) 

The format. Available formats are: CTOR (constructor-like string), LOG (output-ready format) and JSON (struct as JSON).

formatProvider [IFormatProvider](#) 

Unused. Don't bother assigning it anything.

Returns

[string](#) 

The string representation.

Operator operator !=

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

operator !=(Diagnostic, Diagnostic)

Checks if two [Diagnostic](#)s are different from each other.

```
public static bool operator !=(Diagnostic left, Diagnostic right)
```

Parameters

left [Diagnostic](#)

right [Diagnostic](#)

Returns

[bool](#)

True if different.

Operator operator ==

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

operator ==(Diagnostic, Diagnostic)

Checks if two [Diagnostic](#)s are equal to each other.

```
public static bool operator ==(Diagnostic left, Diagnostic right)
```

Parameters

left [Diagnostic](#)

right [Diagnostic](#)

Returns

[bool](#)

True if equals.

Class DiagnosticCollector

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

A list of RSML toolchain errors.

```
public sealed record DiagnosticCollector : IEnumerable<Diagnostic>, IEnumerable, IEquatable<DiagnosticCollector>
```

Inheritance

[object](#)  ← DiagnosticCollector

Implements

[IEnumerable](#)  <[Diagnostic](#)>, [IEnumerable](#) , [IEquatable](#)  <[DiagnosticCollector](#)>

Inherited Members

[object.Equals\(object\)](#) , [object.Equals\(object, object\)](#) , [object.GetHashCode\(\)](#) , [object.GetType\(\)](#) , [object.ReferenceEquals\(object, object\)](#) , [object.ToString\(\)](#) 

Constructors

[DiagnosticCollector\(Severity\)](#)

Creates a new diagnostic collector.

Properties

[HasCriticalErrors](#)

A property that indicates whether there are critical diagnostics. The toolchain should break if this is `true`.

[MinimumCriticalSeverity](#)

The minimum diagnostic severity for a diagnostic to be considered critical.

Methods

[Add\(Diagnostic\)](#)

Adds an error to the list of errors.

[Clear\(\)](#)

Clears the [DiagnosticCollector](#), leaving it fully empty.

[GetAll\(\)](#)

Returns all

[GetEnumerator\(\)](#)

Returns an enumerator that iterates through the collection.

Explicit Interface Implementations

[IEnumerable.GetEnumerator\(\)](#)

Returns an enumerator that iterates through a collection.

Constructor DiagnosticCollector

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

DiagnosticCollector(Severity)

Creates a new diagnostic collector.

```
public DiagnosticCollector(Severity minimumCriticalSeverity = Severity.Error)
```

Parameters

minimumCriticalSeverity [Severity](#)

The minimum diagnostic severity for a diagnostic to be considered critical.

Property HasCriticalErrors

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

HasCriticalErrors

A property that indicates whether there are critical diagnostics. The toolchain should break if this is `true`.

```
public bool HasCriticalErrors { get; }
```

Property Value

[bool](#)

Property MinimumCriticalSeverity

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

MinimumCriticalSeverity

The minimum diagnostic severity for a diagnostic to be considered critical.

```
public Severity MinimumCriticalSeverity { get; }
```

Property Value

[Severity](#)

Method Add

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Add(Diagnostic)

Adds an error to the list of errors.

```
public DiagnosticCollector Add(Diagnostic diagnostic)
```

Parameters

diagnostic [Diagnostic](#)

Returns

[DiagnosticCollector](#)

Method Clear

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Clear()

Clears the [DiagnosticCollector](#), leaving it fully empty.

```
public DiagnosticCollector Clear()
```

Returns

[DiagnosticCollector](#)

Method GetAll

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

GetAll()

Returns all

```
public ImmutableArray<Diagnostic> GetAll()
```

Returns

[ImmutableArray](#)  [<Diagnostic>](#)

Method GetEnumerator

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

GetEnumerator()

Returns an enumerator that iterates through the collection.

```
public IEnumerator<Diagnostic> GetEnumerator()
```

Returns

[IEnumerator](#)  [<Diagnostic>](#)

An enumerator that can be used to iterate through the collection.

Method IEnumerable.GetEnumerator

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

IEnumerable.GetEnumerator()

Returns an enumerator that iterates through a collection.

```
IEnumerator IEnumerable.GetEnumerator()
```

Returns

[IEnumerator](#) 

An [IEnumerator](#)  object that can be used to iterate through the collection.

Enum ErrorCodeCategory

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

The category of an error, which changes its error code.

```
public enum ErrorCodeCategory
```

Fields

General = 0

Any general error code that is not restricted to any specific category.

Internal = 1

An internal error code.

These are usually never thrown unless you implement custom lexers, parsers and whatnot.

Lexer = 2

A lexer error code.

Parser = 3

A parser error code.

Style = 4

A style error code.

Struct ErrorCode

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Represents a code specific to an error.

```
public record struct ErrorCode : IEquatable<ErrorCode>
```

Implements

[IEquatable](#)  <[ErrorCode](#)>

Inherited Members

[ValueType.Equals\(object\)](#)  , [ValueType.GetHashCode\(\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetType\(\)](#)  , [object.ReferenceEquals\(object, object\)](#) 

Constructors

[ErrorCode\(ErrorCategory, int\)](#)

Represents a code specific to an error.

Fields

[LanguagePrefix](#)

The prefix for all error codes related to RSML.

Properties

[Category](#)

The category of the error.

[CategoryPrefix](#)

The specific prefix for each error category.

[Code](#)

The error code.

Methods

[ToString\(\)](#)

Returns the formatted error code.

Constructor ErrorCode

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

ErrorCode(ErrorCategory, int)

Represents a code specific to an error.

```
public ErrorCode(ErrorCategory Category, int Code)
```

Parameters

Category [ErrorCategory](#)

The category of the error.

Code [int](#) 

The error code.

Field LanguagePrefix

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

The prefix for all error codes related to RSML.

```
public const char LanguagePrefix = 'R'
```

Returns

[char](#)

The prefix for all error codes related to RSML.

Property Category

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Category

The category of the error.

```
public ErrorCategory Category { readonly get; set; }
```

Property Value

[ErrorCategory](#).

Property CategoryPrefix

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

CategoryPrefix

The specific prefix for each error category.

```
public readonly char CategoryPrefix { get; }
```

Property Value

[char](#)

Property Code

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Code

The error code.

```
public int Code { readonly get; set; }
```

Property Value

[int](#)

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

ToString()

Returns the formatted error code.

```
public override readonly string ToString()
```

Returns

[string](#) 

The formatted error code.

Struct Result<TValue>

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

An operation's result.

```
public readonly struct Result<TValue>
```

Type Parameters

TValue

The type of the return value when successful.

Inherited Members

[ValueType.Equals\(object\)](#)[↗] , [ValueType.GetHashCode\(\)](#)[↗] , [ValueType.ToString\(\)](#)[↗] ,
[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] ,
[object.ReferenceEquals\(object, object\)](#)[↗]

Properties

[Error](#)

The diagnostic that serves as the error value when an error occurs. Might be null or an arbitrary default if [IsError](#) is **true**.

[IsError](#)

Set to **false** when the operation is successful. Otherwise, it's set to **true**. It also indicates whether [Value](#) can be safely accessed or not (safe to access if [IsError](#) is **false**).

[IsSuccessful](#)

Set to **true** when the operation is successful. Otherwise, it's set to **false**. It also indicates whether [Value](#) can be safely accessed or not (safe to access if [IsSuccessful](#) is **true**).

[Value](#)

The return value. Might be null or an arbitrary default if [IsSuccessful](#) is **false**.

Methods

[Fail\(Diagnostic\)](#)

Creates a return result with an unsuccessful outcome.

[Success\(TValue\)](#)

Creates a return result with a successful outcome.

Property Error

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Error

The diagnostic that serves as the error value when an error occurs. Might be null or an arbitrary default if [IsError](#) is `true`.

```
public Diagnostic Error { get; }
```

Property Value

[Diagnostic](#)

Property IsError

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

IsError

Set to **false** when the operation is successful. Otherwise, it's set to **true**. It also indicates whether [Value](#) can be safely accessed or not (safe to access if [IsError](#) is **false**).

```
public bool IsError { get; }
```

Property Value

[bool](#)

Remarks

[IsSuccessful](#) and [IsError](#) are mutually exclusive conditions. When one is true, the other is false. This means you can use any of the two to evaluate whether the operation was successful and if it's safe to access [Value](#).

Property IsSuccessful

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

IsSuccessful

Set to **true** when the operation is successful. Otherwise, it's set to **false**. It also indicates whether [Value](#) can be safely accessed or not (safe to access if [IsSuccessful](#) is **true**).

```
[MemberNotNullWhen(true, "Value")]  
public bool IsSuccessful { get; }
```

Property Value

[bool](#)

Remarks

[IsSuccessful](#) and [IsError](#) are mutually exclusive conditions. When one is true, the other is false. This means you can use any of the two to evaluate whether the operation was successful and if it's safe to access [Value](#).

Property Value

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Value

The return value. Might be null or an arbitrary default if [IsSuccessful](#) is `false`.

```
[MemberNotNullWhen(true, "Value")]  
public TValue? Value { get; }
```

Property Value

TValue

Method Fail

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Fail(Diagnostic)

Creates a return result with an unsuccessful outcome.

```
public static Result<TValue> Fail(Diagnostic error)
```

Parameters

error [Diagnostic](#)

The error.

Returns

[Result](#)<TValue>

The result.

Method Success

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

Success(TValue)

Creates a return result with a successful outcome.

```
public static Result<TValue> Success(TValue value)
```

Parameters

value TValue

The return value.

Returns

[Result](#)<TValue>

The result.

Enum Severity

Namespace: [OceanApocalypseStudios.RSML.Diagnostics](#)

Assembly: RSML.dll

The severity of an error.

```
public enum Severity : byte
```

Fields

None = 0

No severity information.

Message = 1

Messages and hints.

Warning = 2

Non-critical warnings.

Error = 3

Non-critical errors, such as style errors.

Critical = 4

Fatal error that should abort the executing toolchain component.

Namespace OceanApocalypseStudios. RSML.Execution

Classes

[Interpreter](#)

The base type that deals with evaluating, interpreting and executing RSML.

Class Interpreter

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

The base type that deals with evaluating, interpreting and executing RSML.

```
public abstract class Interpreter : IToolchainComponent, IDisposable
```

Inheritance

[object](#)  ← Interpreter

Implements

[IToolchainComponent](#), [IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Properties

[Configuration](#)

Configurations for the toolchain component.

[IsMutable](#)

Methods

[Dispose\(\)](#)

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

[Freeze\(\)](#)

[Inject\(ToolchainConfiguration\)](#)

Injects a configuration into the toolchain component, modifying it.

Property Configuration

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

Configuration

Configurations for the toolchain component.

```
public ToolchainConfiguration Configuration { get; protected set; }
```

Property Value

[ToolchainConfiguration](#)

Property IsMutable

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

IsMutable

```
public bool IsMutable { get; protected set; }
```

Property Value

[bool](#)

Method Dispose

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public abstract void Dispose()
```

Method Freeze

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

Freeze()

```
public void Freeze()
```

Method Inject

Namespace: [OceanApocalypseStudios.RSML.Execution](#)

Assembly: RSML.dll

Inject(ToolchainConfiguration)

Injects a configuration into the toolchain component, modifying it.

```
public void Inject(ToolchainConfiguration configuration)
```

Parameters

configuration [ToolchainConfiguration](#)

The configuration to inject.

Namespace OceanApocalypseStudios. RSMLExtensibility

Classes

[Class1](#)

Class Class1

Namespace: [OceanApocalypseStudios.RSML.Extensibility](#)

Assembly: RSML.Extensibility.dll

```
public class Class1
```

Inheritance

[object](#)  ← Class1

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Namespace OceanApocalypseStudios. RSMML.Language.Lexing

Classes

[BufferLexer](#)

An implementation of a RSMML lexer backed by a read-only or read-and-write buffer.

[Lexer](#)

The base class for implementations of RSMML lexers and tokenizers.

[StaticLexer](#)

A static lexer for when you don't need to lex from a builder.

[StreamLexer](#)

An implementation of a RSMML lexer backed by a read-only or read-and-write stream.

Structs

[Token](#)

Represents a RSMML token.

Interfaces

[ILexer](#)

Represents a lexer for RSMML.

Class BufferLexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

An implementation of a RSML lexer backed by a read-only or read-and-write buffer.

```
public class BufferLexer : Lexer, ILexer, IToolchainComponent, IDisposable
```

Inheritance

[object](#)  ← [Lexer](#)  ← BufferLexer

Implements

[ILexer](#), [IToolchainComponent](#), [IDisposable](#) 

Inherited Members

[Lexer.Configuration](#) , [Lexer.Inject\(ToolchainConfiguration\)](#) , [Lexer.Dispose\(\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

[BufferLexer\(ReadOnlyBuffer, DiagnosticCollector\)](#)

An implementation of a RSML lexer backed by a read-only or read-and-write buffer.

Methods

[Lex\(\)](#)

Tokenizes a source passed to the lexer.

Constructor BufferLexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

BufferLexer(IReadOnlyBuffer, DiagnosticCollector)

An implementation of a RSML lexer backed by a read-only or read-and-write buffer.

```
public BufferLexer(IReadOnlyBuffer buffer, DiagnosticCollector diagnostics)
```

Parameters

buffer [IReadOnlyBuffer](#)

A buffer. Can be read-only ([IReadOnlyBuffer](#)) or read and write ([IBuffer](#)).

diagnostics [DiagnosticCollector](#)

A collector for all emitted diagnostics.

Method Lex

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Lex()

Tokenizes a source passed to the lexer.

```
public override IEnumerable<Token> Lex()
```

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Interface ILexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Represents a lexer for RSML.

```
public interface ILexer : IToolchainComponent, IDisposable
```

Inherited Members

[IToolchainComponent.Configuration](#) , [IToolchainComponent.Inject\(Configuration\)](#) , [IDisposable.Dispose\(\)](#) 

Remarks

TIP

If you want to add content on top of a lexer, without overriding the extra functionality it adds, you might want to take a look at [Lexer](#).

Methods

[Lex\(\)](#)

Tokenizes a source passed to the lexer.

Method Lex

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Lex()

Tokenizes a source passed to the lexer.

```
IEnumerable<Token> Lex()
```

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Class Lexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

The base class for implementations of RSML lexers and tokenizers.

```
public abstract class Lexer : ILexer, IToolchainComponent, IDisposable
```

Inheritance

[object](#)  ← Lexer

Implements

[ILexer](#), [IToolchainComponent](#), [IDisposable](#) 

Derived

[BufferLexer](#), [StreamLexer](#)

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Properties

[Configuration](#)

Configurations for the toolchain component.

Methods

[Dispose\(\)](#)

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

[Inject\(ToolchainConfiguration\)](#)

Injects a configuration into the toolchain component, modifying it.

[Lex\(\)](#)

Tokenizes a source passed to the lexer.

Property Configuration

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Configuration

Configurations for the toolchain component.

```
public virtual ToolchainConfiguration Configuration { get; protected set; }
```

Property Value

[ToolchainConfiguration](#)

Method Dispose

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public virtual void Dispose()
```

Method Inject

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Inject(ToolchainConfiguration)

Injects a configuration into the toolchain component, modifying it.

```
public virtual void Inject(ToolchainConfiguration configuration)
```

Parameters

configuration [ToolchainConfiguration](#)

The configuration to inject.

Method Lex

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Lex()

Tokenizes a source passed to the lexer.

```
public abstract IEnumerable<Token> Lex()
```

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Class StaticLexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

A static lexer for when you don't need to lex from a builder.

```
public static class StaticLexer
```

Inheritance

[object](#)  ← StaticLexer

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Methods

[Lex\(ISource, ToolchainConfiguration, DiagnosticCollector\)](#)

Tokenizes a source passed to the lexer.

[LexBuffer<TBuffer>\(TBuffer, ToolchainConfiguration, DiagnosticCollector\)](#)

Tokenizes a buffer passed to the lexer.

[LexSpan\(ReadOnlySpanBuffer, ToolchainConfiguration, DiagnosticCollector\)](#)

Tokenizes a read-only span buffer passed to the lexer.

[LexStream<TStream>\(TStream, ToolchainConfiguration, DiagnosticCollector\)](#)

Tokenizes a stream passed to the lexer.

Method Lex

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Lex(ISource, ToolchainConfiguration, DiagnosticCollector)

Tokenizes a source passed to the lexer.

```
public static IEnumerable<Token> Lex(ISource source, ToolchainConfiguration configurations, DiagnosticCollector diagnostics)
```

Parameters

source [ISource](#)

configurations [ToolchainConfiguration](#)

diagnostics [DiagnosticCollector](#)

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Method LexBuffer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

LexBuffer<TBuffer>(TBuffer, ToolchainConfiguration, DiagnosticCollector)

Tokenizes a buffer passed to the lexer.

```
public static IEnumerable<Token> LexBuffer<TBuffer>(TBuffer buffer,  
ToolchainConfiguration configurations, DiagnosticCollector diagnostics) where  
TBuffer : IReadOnlyBuffer, allows ref struct
```

Parameters

buffer TBuffer

configurations [ToolchainConfiguration](#)

diagnostics [DiagnosticCollector](#)

Returns

[IEnumerable](#) [Token](#)

The tokens.

Type Parameters

TBuffer

Remarks

NOTE

This method avoids boxing.

NOTE

This method accepts ref structs if the target framework is .NET 9.0 or higher. Otherwise, you might want to take a look at [LexSpan\(ReadOnlySpanBuffer, Toolchain Configuration, DiagnosticCollector\)](#).

Method LexSpan

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

LexSpan(ReadOnlySpanBuffer, ToolchainConfiguration, DiagnosticCollector)

Tokenizes a read-only span buffer passed to the lexer.

```
public static IEnumerable<Token> LexSpan(ReadOnlySpanBuffer buffer,  
    ToolchainConfiguration configuration, DiagnosticCollector collector)
```

Parameters

buffer [ReadOnlySpanBuffer](#)

configuration [ToolchainConfiguration](#)

collector [DiagnosticCollector](#)

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Method LexStream

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

LexStream<TStream>(TStream, ToolchainConfiguration, DiagnosticCollector)

Tokenizes a stream passed to the lexer.

```
public static IEnumerable<Token> LexStream<TStream>(TStream stream,  
ToolchainConfiguration configurations, DiagnosticCollector diagnostics) where  
TStream : IReadOnlyStream, allows ref struct
```

Parameters

stream TStream

configurations [ToolchainConfiguration](#)

diagnostics [DiagnosticCollector](#)

Returns

[IEnumerable](#) [Token](#)

The tokens.

Type Parameters

TStream

Remarks

NOTE

This method avoids boxing.

Class StreamLexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

An implementation of a RSML lexer backed by a read-only or read-and-write stream.

```
public class StreamLexer : Lexer, ILexer, IToolchainComponent, IDisposable
```

Inheritance

[object](#)  ← [Lexer](#) ← StreamLexer

Implements

[ILexer](#), [IToolchainComponent](#), [IDisposable](#) 

Inherited Members

[Lexer.Configuration](#) , [Lexer.Inject\(ToolchainConfiguration\)](#) , [Lexer.Dispose\(\)](#) , [object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  , [object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  , [object.ToString\(\)](#) 

Constructors

[StreamLexer\(IReadOnlyStream, DiagnosticCollector\)](#)

An implementation of a RSML lexer backed by a read-only or read-and-write stream.

Methods

[Lex\(\)](#)

Tokenizes a source passed to the lexer.

Constructor StreamLexer

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

StreamLexer(IReadOnlyStream, DiagnosticCollector)

An implementation of a RSML lexer backed by a read-only or read-and-write stream.

```
public StreamLexer(IReadOnlyStream stream, DiagnosticCollector diagnostics)
```

Parameters

stream [IReadOnlyStream](#)

A stream. Can be read-only ([IReadOnlyStream](#)) or read and write ([IStream](#)).

diagnostics [DiagnosticCollector](#)

A collector for all emitted diagnostics.

Method Lex

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Lex()

Tokenizes a source passed to the lexer.

```
public override IEnumerable<Token> Lex()
```

Returns

[IEnumerable](#)  <[Token](#)>

The tokens.

Struct Token

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Represents a RSML token.

```
public record struct Token : IEquatable<Token>
```

Implements

[IEquatable](#)  <[Token](#)>

Inherited Members

[ValueType.Equals\(object\)](#)  , [ValueType.GetHashCode\(\)](#)  , [ValueType.ToString\(\)](#)  ,
[object.Equals\(object, object\)](#)  , [object.GetType\(\)](#)  ,
[object.ReferenceEquals\(object, object\)](#) 

Constructors

[Token\(int, object?, SourceSpan\)](#)

Represents a RSML token.

Fields

[Empty](#)

Empty token. Used when something goes wrong.

Properties

[Span](#)

The span where the token occurs.

[TokenKind](#)

An integer that identifies the type of token.

[Value](#)

The token's value.

Constructor Token

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Token(int, object?, SourceSpan)

Represents a RSML token.

```
public Token(int TokenKind, object? Value, SourceSpan Span)
```

Parameters

TokenKind [int](#) 

An integer that identifies the type of token.

Value [object](#) 

The token's value.

Span [SourceSpan](#)

The span where the token occurs.

Field Empty

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Empty token. Used when something goes wrong.

```
public static readonly Token Empty
```

Returns

[Token](#)

Empty token. Used when something goes wrong.

Property Span

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Span

The span where the token occurs.

```
public SourceSpan Span { readonly get; set; }
```

Property Value

[SourceSpan](#)

Property TokenKind

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

TokenKind

An integer that identifies the type of token.

```
public int TokenKind { readonly get; set; }
```

Property Value

[int](#)

Property Value

Namespace: [OceanApocalypseStudios.RSML.Language.Lexing](#)

Assembly: RSML.dll

Value

The token's value.

```
public object? Value { readonly get; set; }
```

Property Value

[object](#)

Namespace OceanApocalypseStudios. RSML.Language.Parsing

Classes

[Parser](#)

The base type that deals with parsing tokens and turning them into an organized tree.

Interfaces

[IParser](#)

Represents a parser for the RSML language.

Interface IParser

Namespace: [OceanApocalypseStudios.RSML.Language.Parsing](#)

Assembly: RSML.dll

Represents a parser for the RSML language.

```
public interface IParser : IToolchainComponent, IDisposable
```

Inherited Members

[IToolchainComponent.Configuration](#) , [IToolchainComponent.Inject\(Configuration\)](#) , [IDisposable.Dispose\(\)](#) 

Class Parser

Namespace: [OceanApocalypseStudios.RSML.Language.Parsing](#)

Assembly: RSML.dll

The base type that deals with parsing tokens and turning them into an organized tree.

```
public abstract class Parser : IParser, IToolchainComponent, IDisposable
```

Inheritance

[object](#)  ← Parser

Implements

[IParser](#), [IToolchainComponent](#), [IDisposable](#) 

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Properties

[Configuration](#)

Configurations for the toolchain component.

Methods

[Dispose\(\)](#)

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

[Inject\(ToolchainConfiguration\)](#)

Injects a configuration into the toolchain component, modifying it.

Property Configuration

Namespace: [OceanApocalypseStudios.RSML.Language.Parsing](#)

Assembly: RSML.dll

Configuration

Configurations for the toolchain component.

```
public ToolchainConfiguration Configuration { get; protected set; }
```

Property Value

[ToolchainConfiguration](#)

Method Dispose

Namespace: [OceanApocalypseStudios.RSML.Language.Parsing](#)

Assembly: RSML.dll

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public abstract void Dispose()
```


Method Inject

Namespace: [OceanApocalypseStudios.RSML.Language.Parsing](#)

Assembly: RSML.dll

Inject(ToolchainConfiguration)

Injects a configuration into the toolchain component, modifying it.

```
public virtual void Inject(ToolchainConfiguration configuration)
```

Parameters

configuration [ToolchainConfiguration](#)

The configuration to inject.

Namespace OceanApocalypseStudios. RSML.Native

Classes

[Class1](#)

Class Class1

Namespace: [OceanApocalypseStudios.RSML.Native](#)

Assembly: RSML.Native.dll

```
public class Class1
```

Inheritance

[object](#)  ← Class1

Inherited Members

[object.Equals\(object\)](#)  , [object.Equals\(object, object\)](#)  , [object.GetHashCode\(\)](#)  ,
[object.GetType\(\)](#)  , [object.MemberwiseClone\(\)](#)  , [object.ReferenceEquals\(object, object\)](#)  ,
[object.ToString\(\)](#) 

Namespace OceanApocalypseStudios. RSMML.Sources

Classes

[ReadOnlyStringBuffer](#)

A read-only buffer backed by a string. All operations opt for performance primarily via the internal use of [ReadOnlySpan<T>](#) over string allocations and also via caching.

Structs

[ReadOnlySpanBuffer](#)

A read-only buffer backed by a span of characters. All operations opt for performance primarily via the internal use of [ReadOnlySpan<T>](#) over string allocations and also via caching.

[SourceLocation](#)

Specifies the location of an item in a [ISource](#).

[SourceSpan](#)

Represents a span taken from a source.

Interfaces

[IBuffer](#)

Represents a read and write buffer of characters.

[IReadOnlyBuffer](#)

Represents a read-only buffer of characters.

[IReadOnlyStream](#)

Represents a read-only sequential stream.

[ISource](#)

A data source for RSMML's toolchain members.

[IStream](#)

Represents a read and write sequential stream.

Interface IBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Represents a read and write buffer of characters.

```
public interface IBuffer : IReadOnlyBuffer, ISource, IDisposable,  
IEquatable<IReadOnlyBuffer?>, IEquatable<char[]?>, IEquatable<string?>,  
IEquatable<ReadOnlyMemory<char>>
```

Inherited Members

[IReadOnlyBuffer.LineCount](#) , [IReadOnlyBuffer.this\[int\]](#) ,
[IReadOnlyBuffer.this\[SourceLocation\]](#) , [IReadOnlyBuffer.this\[SourceSpan\]](#) ,
[IReadOnlyBuffer.CountUntilEndOfLine\(int, out bool\)](#) ,
[IReadOnlyBuffer.CountUntilNotWhitespace\(int\)](#) ,
[IReadOnlyBuffer.CountUntilWhitespace\(int\)](#) ,
[IReadOnlyBuffer.CountWhile\(Func<int, char, bool>, int\)](#) ,
[IReadOnlyBuffer.GetLengthOfLine\(int\)](#) , [IReadOnlyBuffer.GetLengthOfLineFromIndex\(int\)](#) ,
[IReadOnlyBuffer.GetLine\(int\)](#) , [IReadOnlyBuffer.GetLineFromIndex\(int\)](#) ,
[IReadOnlyBuffer.GetLineAsSpan\(int\)](#) , [IReadOnlyBuffer.GetLineAsSpanFromIndex\(int\)](#) ,
[IReadOnlyBuffer.GetLineNumberFromIndex\(int\)](#) , [IReadOnlyBuffer.GetSourceLocation\(int\)](#) ,
[IReadOnlyBuffer.GetSourceSpan\(int, int\)](#) , [IReadOnlyBuffer.Slice\(int, int\)](#) ,
[IReadOnlyBuffer.TrySlice\(int, Span<char>\)](#) ,
[IReadOnlyBuffer.TrySlice\(SourceSpan, Span<char>\)](#) ,
[IReadOnlyBuffer.TryGetChar\(int, out char\)](#) ,
[IReadOnlyBuffer.TryGetChar\(SourceLocation, out char\)](#) ,
[IReadOnlyBuffer.TryGetLine\(int, Span<char>\)](#) ,
[IReadOnlyBuffer.TryGetLineFromIndex\(int, Span<char>\)](#) , [ISource.IsEmpty](#) ,
[ISource.IsReadOnly](#) , [ISource.Length](#) , [IDisposable.Dispose\(\)](#)  ,
[IEquatable<IReadOnlyBuffer>.Equals\(IReadOnlyBuffer\)](#)  ,
[IEquatable<char\[\]>.Equals\(char\[\]\)](#)  , [IEquatable<string>.Equals\(string\)](#)  ,
[IEquatable<ReadOnlyMemory<char>>.Equals\(ReadOnlyMemory<char>\)](#) 

Interface IReadOnlyBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Represents a read-only buffer of characters.

```
public interface IReadOnlyBuffer : ISource, IDisposable, IEquatable<IReadOnlyBuffer?>, IEquatable<char[]?>, IEquatable<string?>, IEquatable<ReadOnlyMemory<char>>
```

Inherited Members

[ISource.IsEmpty](#) , [ISource.IsReadOnly](#) , [ISource.Length](#) , [IDisposable.Dispose\(\)](#) , [IEquatable<IReadOnlyBuffer>.Equals\(IReadOnlyBuffer\)](#) , [IEquatable<char\[\]>.Equals\(char\[\]\)](#) , [IEquatable<string>.Equals\(string\)](#) , [IEquatable<ReadOnlyMemory<char>>.Equals\(ReadOnlyMemory<char>\)](#)

Properties

[this\[SourceLocation\]](#)

Gets a single item out of the buffer.

[this\[SourceSpan\]](#)

Gets a range of items out of the buffer.

[this\[int\]](#)

Gets a single item out of the buffer.

[LineCount](#)

The total amount of lines in the buffer.

Methods

[CountUntilEndOfLine\(int, out bool\)](#)

Counts the amount of items until the next line separator in the buffer, relative to a given **index**. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

[CountUntilNotWhitespace\(int\)](#)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[`CountUntilWhitespace\(int\)`](#)

Counts the amount of items until the next whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[`CountWhile\(Func<int, char, bool>, int\)`](#)

Counts the amount of items, starting from a given `index`, while a `predicate` returns `true`.

[`GetLengthOfLine\(int\)`](#)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

[`GetLengthOfLineFromIndex\(int\)`](#)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

[`GetLine\(int\)`](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[`GetLineAsSpan\(int\)`](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[`GetLineAsSpanFromIndex\(int\)`](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[`GetLineFromIndex\(int\)`](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[`GetLineNumberFromIndex\(int\)`](#)

Determines the 0-based line number of the line that contains the item located at `index`.

[`GetSourceLocation\(int\)`](#)

Converts an index into a location.

[`GetSourceSpan\(int, int\)`](#)

Converts the buffer region into a span.

[`Slice\(int, int\)`](#)

Slices a region of the buffer.

[`TryGetChar\(SourceLocation, out char\)`](#)

Tries to return the item at the specified `location`.

[TryGetChar\(int, out char\)](#)

Tries to return the item at `index`.

[TryGetLine\(int, Span<char>\)](#)

Given a 0-based line number, assigns the exact line to a result buffer (`destination`). No end of line characters are added.

[TryGetLineFromIndex\(int, Span<char>\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[TrySlice\(SourceSpan, Span<char>\)](#)

Slices a region of the buffer into a performant span.

[TrySlice\(int, Span<char>\)](#)

Slices a region of the buffer into a performant span.

Property LineCount

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

LineCount

The total amount of lines in the buffer.

```
int LineCount { get; }
```

Property Value

[int](#)

Remarks

Keep in mind lines might be empty.

Property this

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

this[int]

Gets a single item out of the buffer.

```
char? this[int index] { get; }
```

Parameters

index [int](#)

The index of the item to retrieve.

Property Value

[char](#)?

The item.

this[SourceLocation]

Gets a single item out of the buffer.

```
char? this[SourceLocation location] { get; }
```

Parameters

location [SourceLocation](#)

The location of the item to retrieve.

Property Value

[char](#)? 

The item.

this[SourceSpan]

Gets a range of items out of the buffer.

```
Result<string> this[SourceSpan span] { get; }
```

Parameters

span [SourceSpan](#)

The span of items to get.

Property Value

[Result](#)<[string](#)  >

The items.

Method CountUntilEndOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilEndOfLine(int, out bool)

Counts the amount of items until the next line separator in the buffer, relative to a given **index**. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

```
Result<int> CountUntilEndOfLine(int index, out bool isCrLf)
```

Parameters

index [int](#)

The index at which to start counting.

isCrLf [bool](#)

Whether the line separator at which the method stopped is the CR in a CRLF sequence. If true, the next item in the buffer is LF.

Returns

[Result](#)<[int](#)>

The index of the next line separator, relative to an **index**.

Method CountUntilNotWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilNotWhitespace(int)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

```
Result<int> CountUntilNotWhitespace(int index)
```

Parameters

`index` [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next non-whitespace item, relative to a `index`.

Method CountUntilWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilWhitespace(int)

Counts the amount of items until the next whitespace item in the buffer, relative to a given **index**. Line separators are included in the whitespace category.

```
Result<int> CountUntilWhitespace(int index)
```

Parameters

index [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next whitespace item, relative to a **index**.

Method CountWhile

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountWhile(Func<int, char, bool>, int)

Counts the amount of items, starting from a given **index**, while a **predicate** returns **true**.

```
Result<int> CountWhile(Func<int, char, bool> predicate, int index)
```

Parameters

predicate [Func](#)<[int](#), [char](#), [bool](#)>

A function that takes the current index (relative to **index**), which is incremented every item, and the item associated with it. Execution stops when the predicate returns **false** or the index is out of bounds.

index [int](#)

The index at which to start counting; all indexes will also be given to the **predicate** as an offset that when added to the index of the position equal the actual index.

Returns

[Result](#)<[int](#)>

The amount of items counted.

Method GetLengthOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLine(int)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

```
Result<int> GetLengthOfLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result](#)<[int](#)>

The length of the line.

Method GetLengthOfLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLineFromIndex(int)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

```
Result<int> GetLengthOfLineFromIndex(int index)
```

Parameters

index [int](#)

The 0-based index whose line is considered.

Returns

[Result](#)<[int](#)>

The length of the line.

Method GetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLine(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
Result<string> GetLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result](#)<[string](#)>

The line as an array of items.

Method GetLineAsSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpan(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
ReadOnlySpan<char> GetLineAsSpan(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[ReadOnlySpan](#) <[char](#)>

The line as a span.

Method GetLineAsSpanFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpanFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
ReadOnlySpan<char> GetLineAsSpanFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[ReadOnlySpan](#) <[char](#)>

The line, as a span.

Method GetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
Result<string> GetLineFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[Result](#)<[string](#)>

The line, as an array of items.

Method GetLineNumberFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineNumberFromIndex(int)

Determines the 0-based line number of the line that contains the item located at `index`.

```
Result<int> GetLineNumberFromIndex(int index)
```

Parameters

`index` [int](#)

The index whose parent line's number is to be returned.

Returns

[Result](#)<[int](#)>

The 0-based number of the line that contains item located at `index`.

Method GetSourceLocation

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceLocation(int)

Converts an index into a location.

```
Result<SourceLocation> GetSourceLocation(int index)
```

Parameters

index [int](#)

The index.

Returns

[Result](#)<[SourceLocation](#)>

The location.

Method GetSourceSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceSpan(int, int)

Converts the buffer region into a span.

```
Result<SourceSpan> GetSourceSpan(int startIndex, int endIndex)
```

Parameters

startIndex [int](#)

The starting index.

endIndex [int](#)

The end index, which is included in the span.

Returns

[Result](#)<[SourceSpan](#)>

The span.

Method Slice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Slice(int, int)

Slices a region of the buffer.

```
Result<string> Slice(int start, int length)
```

Parameters

start [int](#)

The index of the first item in the slice.

length [int](#)

The amount of items to slice starting at **start**.

Returns

[Result](#)<[string](#)>

A slice, as an array of items.

Method TryGetChar

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetChar(int, out char)

Tries to return the item at `index`.

```
bool TryGetChar(int index, out char item)
```

Parameters

`index` [int](#)

The index of the character.

`item` [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

TryGetChar(SourceLocation, out char)

Tries to return the item at the specified `location`.

```
bool TryGetChar(SourceLocation location, out char item)
```

Parameters

`location` [SourceLocation](#)

The item's location.

item [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

Method TryGetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLine(int, Span<char>)

Given a 0-based line number, assigns the exact line to a result buffer (*destination*). No end of line characters are added.

```
bool TryGetLine(int lineNumber, Span<char> destination)
```

Parameters

lineNumber [int](#)

The 0-based line number.

destination [Span](#)<[char](#)>

The destination buffer for the line.

Returns

[bool](#)

True if successful.

Method TryGetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLineFromIndex(int, Span<char>)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
bool TryGetLineFromIndex(int index, Span<char> destination)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

`destination` [Span](#)<[char](#)>

The destination span that will contain the line.

Returns

[bool](#)

True if successful.

Method TrySlice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TrySlice(int, Span<char>)

Slices a region of the buffer into a performant span.

```
bool TrySlice(int start, Span<char> slice)
```

Parameters

start [int](#)

The index of the first item in the slice.

slice [Span](#) <[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#)

TrySlice(SourceSpan, Span<char>)

Slices a region of the buffer into a performant span.

```
bool TrySlice(SourceSpan sourceSpan, Span<char> slice)
```

Parameters

sourceSpan [SourceSpan](#)

The span indicating what the slice is.

slice [Span](#) <[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#) 

Interface IReadOnlyStream

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Represents a read-only sequential stream.

```
public interface IReadOnlyStream : ISource, IDisposable, IEquatable<IReadOnlyStream?>
```

Inherited Members

[ISource.IsEmpty](#) , [ISource.IsReadOnly](#) , [ISource.Length](#) , [IDisposable.Dispose\(\)](#) , [IEquatable<IReadOnlyStream>.Equals\(IReadOnlyStream\)](#)

Properties

[CursorIndex](#)

The current index of the cursor.

Property CursorIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CursorIndex

The current index of the cursor.

```
int CursorIndex { get; }
```

Property Value

[int](#)

Interface ISource

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

A data source for RSML's toolchain members.

```
public interface ISource : IDisposable
```

Inherited Members

[IDisposable.Dispose\(\)](#) 

Properties

[IsEmpty](#)

Whether the source is completely empty.

[IsReadOnly](#)

Whether the source can be mutated.

[Length](#)

The length of the source.

Property IsEmpty

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsEmpty

Whether the source is completely empty.

```
bool IsEmpty { get; }
```

Property Value

[bool](#) 

Property IsReadOnly

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsReadOnly

Whether the source can be mutated.

```
bool IsReadOnly { get; }
```

Property Value

[bool](#) 

Property Length

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Length

The length of the source.

```
int Length { get; }
```

Property Value

[int](#)

Interface IStream

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Represents a read and write sequential stream.

```
public interface IStream : IReadOnlyStream, ISource, IDisposable,  
    IEquatable<IReadOnlyStream?>
```

Inherited Members

[IReadOnlyStream.CursorIndex](#) , [ISource.IsEmpty](#) , [ISource.IsReadOnly](#) , [ISource.Length](#) ,
[IDisposable.Dispose\(\)](#) , [IEquatable<IReadOnlyStream>.Equals\(IReadOnlyStream\)](#)

Struct ReadOnlySpanBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

A read-only buffer backed by a span of characters. All operations opt for performance primarily via the internal use of [ReadOnlySpan<T>](#) over string allocations and also via caching.

```
public ref struct ReadOnlySpanBuffer : IReadOnlyBuffer, ISource, IDisposable,
    IEquatable<IReadOnlyBuffer?>, IEquatable<char[]?>, IEquatable<string?>,
    IEquatable<ReadOnlyMemory<char>>, ISupportsCache
```

Implements

[IReadOnlyBuffer](#), [ISource](#), [IDisposable](#), [IEquatable](#)<[IReadOnlyBuffer](#)>, [IEquatable](#)<[char](#)[]>, [IEquatable](#)<[string](#)>, [IEquatable](#)<[ReadOnlyMemory](#)<[char](#)>>, [ISupportsCache](#)

Inherited Members

[object.Equals\(object, object\)](#), [object.GetType\(\)](#), [object.ReferenceEquals\(object, object\)](#)

Remarks

TIP

The main advantage of this type over [ReadOnlyStringBuffer](#) is that you don't need to allocate a class and, on initialization, you don't need to allocate a string.

Constructors

[ReadOnlySpanBuffer\(char\[\]\)](#)

Initializes a new [ReadOnlySpanBuffer](#) with an array of characters.

[ReadOnlySpanBuffer\(ReadOnlySpan<char>\)](#)

Initializes a new [ReadOnlySpanBuffer](#) with a span.

[ReadOnlySpanBuffer\(string\)](#)

Initializes a new [ReadOnlySpanBuffer](#) with a string.

Properties

[CacheExists](#)

Whether there's cached data.

[IsEmpty](#)

Whether the source is completely empty.

[IsReadOnly](#)

Whether the source can be mutated.

[this\[SourceLocation\]](#)

Gets a single item out of the buffer.

[this\[SourceSpan\]](#)

Gets a range of items out of the buffer.

[this\[int\]](#)

Gets a single item out of the buffer.

[Length](#)

The length of the source.

[LineCount](#)

The total amount of lines in the buffer.

Methods

[BuildCache\(\)](#)

Builds the cache if it doesn't exist yet.

[BuildCache\(bool\)](#)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

[CountUntilEndOfLine\(int, out bool\)](#)

Counts the amount of items until the next line separator in the buffer, relative to a given `index`. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

[CountUntilNotWhitespace\(int\)](#)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[CountUntilWhitespace\(int\)](#)

Counts the amount of items until the next whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[CountWhile\(Func<int, char, bool>, int\)](#)

Counts the amount of items, starting from a given `index`, while a `predicate` returns `true`.

[Dispose\(\)](#)

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

[Equals\(IReadOnlyBuffer?\)](#)

Checks if another read-only buffer is equal to the current instance.

[Equals\(ReadOnlySpanBuffer\)](#)

Checks if another read-only span buffer is equal to the current instance.

[Equals\(char\[\]?\)](#)

Checks if an array of characters is equal to the current instance.

[Equals\(object?\)](#)

Indicates whether this instance and a specified object are equal.

[Equals\(ReadOnlyMemory<char>\)](#)

Checks if a read-only contiguous region of memory is equal to the current instance.

[Equals\(ReadOnlySpan<char>\)](#)

Checks if a read-only contiguous region of memory is equal to the current instance.

[Equals\(string?\)](#)

Checks if a string is equal to the current instance.

[GetHashCode\(\)](#)

Returns the hash code for this instance.

[GetLengthOfLine\(int\)](#)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

[GetLengthOfLineFromIndex\(int\)](#)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

[GetLine\(int\)](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[GetLineAsSpan\(int\)](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[GetLineAsSpanFromIndex\(int\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[GetLineFromIndex\(int\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[GetLineNumberFromIndex\(int\)](#)

Determines the 0-based line number of the line that contains the item located at `index`.

[GetSourceLocation\(int\)](#)

Converts an index into a location.

[GetSourceSpan\(int, int\)](#)

Converts the buffer region into a span.

[Slice\(int, int\)](#)

Slices a region of the buffer.

[ToString\(\)](#)

Returns the buffer's content as a [string](#).

[TryGetChar\(SourceLocation, out char\)](#)

Tries to return the item at the specified `location`.

[TryGetChar\(int, out char\)](#)

Tries to return the item at `index`.

[TryGetLine\(int, scoped Span<char>\)](#)

Given a 0-based line number, assigns the exact line to a result buffer (`destination`). No end of line characters are added.

[TryGetLineFromIndex\(int, scoped Span<char>\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[TrySlice\(SourceSpan, Span<char>\)](#)

Slices a region of the buffer into a performant span.

[TrySlice\(int, Span<char>\)](#)

Slices a region of the buffer into a performant span.

Operators

[operator ==\(ReadOnlySpanBuffer, ReadOnlySpanBuffer\)](#)

Checks if two read-only span buffers are equals.

[operator !=\(ReadOnlySpanBuffer, ReadOnlySpanBuffer\)](#)

Checks if two read-only span buffers are different.

Constructor ReadOnlySpanBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ReadOnlySpanBuffer(string)

Initializes a new [ReadOnlySpanBuffer](#) with a string.

```
public ReadOnlySpanBuffer(string content)
```

Parameters

content [string](#) 

The string that the buffer will wrap.

ReadOnlySpanBuffer(ReadOnlySpan<char>)

Initializes a new [ReadOnlySpanBuffer](#) with a span.

```
public ReadOnlySpanBuffer(ReadOnlySpan<char> content)
```

Parameters

content [ReadOnlySpan](#)  <[char](#)  >

The span pointing to the string's data.

ReadOnlySpanBuffer(char[])

Initializes a new [ReadOnlySpanBuffer](#) with an array of characters.

```
public ReadOnlySpanBuffer(char[] content)
```

Parameters

`content` [char](#)[↗][]

The array of characters to use for the buffer.

Property CacheExists

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CacheExists

Whether there's cached data.

```
public readonly bool CacheExists { get; }
```

Property Value

[bool](#)

Property IsEmpty

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsEmpty

Whether the source is completely empty.

```
public readonly bool IsEmpty { get; }
```

Property Value

[bool](#)

Property IsReadOnly

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsReadOnly

Whether the source can be mutated.

```
public readonly bool IsReadOnly { get; }
```

Property Value

[bool](#)

Remarks

Always returns **true**, as [ReadOnlySpanBuffer](#) only supports read-only content (hence the name).

Property Length

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Length

The length of the source.

```
public readonly int Length { get; }
```

Property Value

[int](#)

Property LineCount

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

LineCount

The total amount of lines in the buffer.

```
public int LineCount { get; }
```

Property Value

[int](#)

Remarks

[LineCount](#) automatically builds cache if no cached data exists. No [BuildCache\(\)](#) calls are necessary.

Property this

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

this[int]

Gets a single item out of the buffer.

```
public readonly char? this[int index] { get; }
```

Parameters

index [int](#)

The index of the item to retrieve.

Property Value

[char](#)?

The item.

this[SourceLocation]

Gets a single item out of the buffer.

```
public readonly char? this[SourceLocation location] { get; }
```

Parameters

location [SourceLocation](#)

The location of the item to retrieve.

Property Value

[char](#)[↗]?

The item.

this[SourceSpan]

Gets a range of items out of the buffer.

```
public readonly Result<string> this[SourceSpan span] { get; }
```

Parameters

span [SourceSpan](#)

The span of items to get.

Property Value

[Result](#)<[string](#)[↗]>

The items.

Method BuildCache

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

BuildCache()

Builds the cache if it doesn't exist yet.

```
public void BuildCache()
```

BuildCache(bool)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

```
public void BuildCache(bool forceRebuild)
```

Parameters

`forceRebuild` [bool](#)

Whether to force the cache to be built even if it exists.

Method CountUntilEndOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilEndOfLine(int, out bool)

Counts the amount of items until the next line separator in the buffer, relative to a given **index**. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

```
public Result<int> CountUntilEndOfLine(int index, out bool isCrLf)
```

Parameters

index [int](#)

The index at which to start counting.

isCrLf [bool](#)

Whether the line separator at which the method stopped is the CR in a CRLF sequence. If true, the next item in the buffer is LF.

Returns

[Result](#)<[int](#)>

The index of the next line separator, relative to an **index**.

Remarks

NOTE

This method allows the EOF index as in-range. The convention is as follows:

- If the index is EOF ([Length](#)), then the output is always 0 and **isCrLf** is always **false**.
- If the index is the last ([Length](#) - 1), then the output is always 0.

i NOTE

`isCrLf` is only `true` if all the following conditions are true:

- The next line start counting from `index` is preceded by a CRLF sequence.
- `index` does not point to the LF in the CRLF sequence.
- `index` does not point to EOF.

Method CountUntilNotWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilNotWhitespace(int)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

```
public readonly Result<int> CountUntilNotWhitespace(int index)
```

Parameters

`index` [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next non-whitespace item, relative to a `index`.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that is not whitespace, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that are not whitespace), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method CountUntilWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilWhitespace(int)

Counts the amount of items until the next whitespace item in the buffer, relative to a given **index**. Line separators are included in the whitespace category.

```
public readonly Result<int> CountUntilWhitespace(int index)
```

Parameters

index [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next whitespace item, relative to a **index**.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that is whitespace, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that are whitespace), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method CountWhile

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountWhile(Func<int, char, bool>, int)

Counts the amount of items, starting from a given **index**, while a **predicate** returns **true**.

```
public readonly Result<int> CountWhile(Func<int, char, bool> predicate, int index)
```

Parameters

predicate [Func](#)<[int](#), [char](#), [bool](#)>

A function that takes the current index (relative to **index**), which is incremented every item, and the item associated with it. Execution stops when the predicate returns **false** or the index is out of bounds.

index [int](#)

The index at which to start counting; all indexes will also be given to the **predicate** as an offset that when added to the index of the position equal the actual index.

Returns

[Result](#)<[int](#)>

The amount of items counted.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that fails to verify the `predicate`, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that fail to verify the `predicate`), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method Dispose

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public readonly void Dispose()
```

Method Equals

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override readonly bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current instance.

Returns

[bool](#)

[true](#) if **obj** and this instance are the same type and represent the same value; otherwise, [false](#).

Equals(char[]?)

Checks if an array of characters is equal to the current instance.

```
public readonly bool Equals(char[]? other)
```

Parameters

other [char](#)[]

The array.

Returns

[bool](#)

True if equals.

Equals(IReadOnlyBuffer?)

Checks if another read-only buffer is equal to the current instance.

```
public readonly bool Equals(IReadOnlyBuffer? other)
```

Parameters

other [IReadOnlyBuffer](#)

The other read-only buffer.

Returns

[bool](#)

True if equals.

Equals(ReadOnlyMemory<char>)

Checks if a read-only contiguous region of memory is equal to the current instance.

```
public readonly bool Equals(ReadOnlyMemory<char> other)
```

Parameters

other [ReadOnlyMemory](#) <[char](#)>

The region of memory.

Returns

[bool](#)

True if equals.

Equals(ReadOnlySpan<char>)

Checks if a read-only contiguous region of memory is equal to the current instance.

```
public readonly bool Equals(ReadOnlySpan<char> other)
```

Parameters

other [ReadOnlySpan](#) <[char](#)>

The region of memory.

Returns

[bool](#)

True if equals.

Equals(ReadOnlySpanBuffer)

Checks if another read-only span buffer is equal to the current instance.

```
public readonly bool Equals(ReadOnlySpanBuffer other)
```

Parameters

other [ReadOnlySpanBuffer](#)

The other read-only span buffer.

Returns

[bool](#)

True if equals.

Equals(string?)

Checks if a string is equal to the current instance.

```
public readonly bool Equals(string? other)
```

Parameters

other [string](#) 

The string.

Returns

[bool](#) 

True if equals.

Method GetHashCode

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

[int](#)

A 32-bit signed integer that is the hash code for this instance.

Method GetLengthOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLine(int)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

```
public Result<int> GetLengthOfLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result<int>](#)

The length of the line.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLengthOfLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLineFromIndex(int)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

```
public Result<int> GetLengthOfLineFromIndex(int index)
```

Parameters

index [int](#)

The 0-based index whose line is considered.

Returns

[Result](#)<[int](#)>

The length of the line.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLine(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
public Result<string> GetLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result](#)<[string](#)>

The line as an array of items.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLineAsSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpan(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
public ReadOnlySpan<char> GetLineAsSpan(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[ReadOnlySpan](#) <[char](#)>

The line as a span.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLineAsSpanFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpanFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public ReadOnlySpan<char> GetLineAsSpanFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[ReadOnlySpan](#) <[char](#)>

The line, as a span.

Method GetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public Result<string> GetLineFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[Result](#)<[string](#)>

The line, as an array of items.

Method GetLineNumberFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineNumberFromIndex(int)

Determines the 0-based line number of the line that contains the item located at `index`.

```
public Result<int> GetLineNumberFromIndex(int index)
```

Parameters

`index` [int](#)

The index whose parent line's number is to be returned.

Returns

[Result](#)<[int](#)>

The 0-based number of the line that contains item located at `index`.

Remarks

NOTE

This method follows EOF conventions.

Method GetSourceLocation

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceLocation(int)

Converts an index into a location.

```
public Result<SourceLocation> GetSourceLocation(int index)
```

Parameters

index [int](#)

The index.

Returns

[Result](#)<[SourceLocation](#)>

The location.

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlySpanBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered a location.

Method GetSourceSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceSpan(int, int)

Converts the buffer region into a span.

```
public Result<SourceSpan> GetSourceSpan(int startIndex, int endIndex)
```

Parameters

startIndex [int](#)

The starting index.

endIndex [int](#)

The end index, which is included in the span.

Returns

[Result](#)<[SourceSpan](#)>

The span.

Method Slice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Slice(int, int)

Slices a region of the buffer.

```
public readonly Result<string> Slice(int start, int length)
```

Parameters

start [int](#)

The index of the first item in the slice.

length [int](#)

The amount of items to slice starting at **start**.

Returns

[Result](#)<[string](#)>

A slice, as an array of items.

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlySpanBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ToString()

Returns the buffer's content as a [string](#) .

```
public override readonly string ToString()
```

Returns

[string](#) 

The buffer's content.

Method TryGetChar

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetChar(int, out char)

Tries to return the item at `index`.

```
public readonly bool TryGetChar(int index, out char item)
```

Parameters

`index` [int](#)

The index of the character.

`item` [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

Remarks

NOTE

This method follows the EOF convention where the EOF character is 0 (`'\0'`) and the return value is `false`, due to EOF not being an actual buffer location.

TryGetChar(SourceLocation, out char)

Tries to return the item at the specified `location`.

```
public readonly bool TryGetChar(SourceLocation location, out char item)
```

Parameters

location [SourceLocation](#)

The item's location.

item [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

Remarks

NOTE

This method follows the EOF convention where the EOF character is 0 ('`\0`') and the return value is `false`, due to EOF not being an actual buffer location.

Method TryGetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLine(int, scoped Span<char>)

Given a 0-based line number, assigns the exact line to a result buffer (*destination*). No end of line characters are added.

```
public bool TryGetLine(int lineNumber, scoped Span<char> destination)
```

Parameters

lineNumber [int](#)

The 0-based line number.

destination [Span](#) <[char](#)>

The destination buffer for the line.

Returns

[bool](#)

True if successful.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method TryGetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLineFromIndex(int, scoped Span<char>)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public bool TryGetLineFromIndex(int index, scoped Span<char> destination)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

`destination` [Span](#) <[char](#)>

The destination span that will contain the line.

Returns

[bool](#)

True if successful.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1). If `index` is EOF, the line will also be EOF.

Method TrySlice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TrySlice(int, Span<char>)

Slices a region of the buffer into a performant span.

```
public readonly bool TrySlice(int start, Span<char> slice)
```

Parameters

start [int](#)

The index of the first item in the slice.

slice [Span](#) <[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#)

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlySpanBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

TrySlice(SourceSpan, Span<char>)

Slices a region of the buffer into a performant span.

```
public readonly bool TrySlice(SourceSpan sourceSpan, Span<char> slice)
```

Parameters

sourceSpan [SourceSpan](#)

The span indicating what the slice is.

slice [Span](#) <[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#)

Remarks

⊗ **IMPORTANT**

Unlike with other [ReadOnlySpanBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

Operator operator !=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator !=(ReadOnlySpanBuffer, ReadOnlySpanBuffer)

Checks if two read-only span buffers are different.

```
public static bool operator !=(ReadOnlySpanBuffer left, ReadOnlySpanBuffer right)
```

Parameters

left [ReadOnlySpanBuffer](#)

right [ReadOnlySpanBuffer](#)

Returns

[bool](#)

True if different.

Operator operator ==

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator ==(ReadOnlySpanBuffer, ReadOnlySpanBuffer)

Checks if two read-only span buffers are equals.

```
public static bool operator ==(ReadOnlySpanBuffer left, ReadOnlySpanBuffer right)
```

Parameters

left [ReadOnlySpanBuffer](#)

right [ReadOnlySpanBuffer](#)

Returns

[bool](#) 

True if equals.

Class ReadOnlyStringBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

A read-only buffer backed by a string. All operations opt for performance primarily via the internal use of [ReadOnlySpan<T>](#) over string allocations and also via caching.

```
public class ReadOnlyStringBuffer : IReadOnlyBuffer, ISource, IDisposable,
    IEquatable<IReadOnlyBuffer?>, IEquatable<char[]?>, IEquatable<ReadOnlyMemory<char>>,>,
    ISupportsCache, IEquatable<string?>
```

Inheritance

[object](#) ← [ReadOnlyStringBuffer](#)

Implements

[IReadOnlyBuffer](#), [ISource](#), [IDisposable](#), [IEquatable](#)<[IReadOnlyBuffer](#)>, [IEquatable](#)<[char](#)[]>, [IEquatable](#)<[ReadOnlyMemory](#)<[char](#)>>, [ISupportsCache](#), [IEquatable](#)<[string](#)>

Inherited Members

[object.Equals\(object, object\)](#), [object.GetType\(\)](#), [object.MemberwiseClone\(\)](#), [object.ReferenceEquals\(object, object\)](#)

Remarks

TIP

If you wish to avoid allocating this buffer, it's recommended to take a look at [ReadOnlySpanBuffer](#).

Constructors

[ReadOnlyStringBuffer\(byte*, int, Encoding?\)](#)

Initializes a new [ReadOnlyStringBuffer](#) with a pointer referencing an array of bytes and the encoding to use when decoding them.

[ReadOnlyStringBuffer\(byte\[\], Encoding?\)](#)

Initializes a new [ReadOnlyStringBuffer](#) with an array of bytes and the encoding to use when decoding them.

[ReadOnlyStringBuffer\(char\[\]\)](#)

Initializes a new [ReadOnlyStringBuffer](#) with an array of characters.

[ReadOnlyStringBuffer\(ReadOnlySpan<char>\)](#)

Initializes a new [ReadOnlyStringBuffer](#) by allocating a string from a [ReadOnlySpan<T>](#) .

[ReadOnlyStringBuffer\(string\)](#)

Initializes a new [ReadOnlyStringBuffer](#) with a string.

Properties

[CacheExists](#)

Whether there's cached data.

[IsEmpty](#)

Whether the source is completely empty.

[IsReadOnly](#)

Whether the source can be mutated.

[this\[SourceLocation\]](#)

Gets a single item out of the buffer.

[this\[SourceSpan\]](#)

Gets a range of items out of the buffer.

[this\[int\]](#)

Gets a single item out of the buffer.

[Length](#)

The length of the source.

[LineCount](#)

The total amount of lines in the buffer.

Methods

[BuildCache\(\)](#)

Builds the cache if it doesn't exist yet.

[BuildCache\(bool\)](#)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

[CountUntilEndOfLine\(int, out bool\)](#)

Counts the amount of items until the next line separator in the buffer, relative to a given `index`. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

[CountUntilNotWhitespace\(int\)](#)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[CountUntilWhitespace\(int\)](#)

Counts the amount of items until the next whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

[CountWhile\(Func<int, char, bool>, int\)](#)

Counts the amount of items, starting from a given `index`, while a `predicate` returns `true`.

[Dispose\(\)](#)

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

[Equals\(IReadOnlyBuffer?\)](#)

Checks if another read-only buffer is equal to the current instance.

[Equals\(char\[\]?\)](#)

Checks if an array of characters is equal to the current instance.

[Equals\(object?\)](#)

Determines whether the specified object is equal to the current object.

[Equals\(ReadOnlyMemory<char>\)](#)

Checks if a read-only contiguous region of memory is equal to the current instance.

[Equals\(ReadOnlySpan<char>\)](#)

Checks if a read-only contiguous region of memory is equal to the current instance.

[Equals\(string?\)](#)

Checks if a string is equal to the current instance.

[GetHashCode\(\)](#)

Serves as the default hash function.

[GetLengthOfLine\(int\)](#)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

[GetLengthOfLineFromIndex\(int\)](#)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

[GetLine\(int\)](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[GetLineAsSpan\(int\)](#)

Given a 0-based line number, returns the matching line as an array of buffer items.

[GetLineAsSpanFromIndex\(int\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[GetLineFromIndex\(int\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[GetLineNumberFromIndex\(int\)](#)

Determines the 0-based line number of the line that contains the item located at `index`.

[GetSourceLocation\(int\)](#)

Converts an index into a location.

[GetSourceSpan\(int, int\)](#)

Converts the buffer region into a span.

[Slice\(int, int\)](#)

Slices a region of the buffer.

[ToString\(\)](#)

Returns the buffer's content as a [string](#).

[TryGetChar\(SourceLocation, out char\)](#)

Tries to return the item at the specified `location`.

[TryGetChar\(int, out char\)](#)

Tries to return the item at `index`.

[TryGetLine\(int, Span<char>\)](#)

Given a 0-based line number, assigns the exact line to a result buffer (`destination`). No end of line characters are added.

[TryGetLineFromIndex\(int, Span<char>\)](#)

Tries to read the line that contains the item at `index`. No end of line characters are added.

[TrySlice\(SourceSpan, Span<char>\)](#)

Slices a region of the buffer into a performant span.

[TrySlice\(int, Span<char>\)](#)

Slices a region of the buffer into a performant span.

Operators

[operator ==\(ReadOnlyStringBuffer, ReadOnlyStringBuffer\)](#)

Checks if two read-only string buffers are equals.

[operator !=\(ReadOnlyStringBuffer, ReadOnlyStringBuffer\)](#)

Checks if two read-only string buffers are different.

Constructor ReadOnlyStringBuffer

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ReadOnlyStringBuffer(string)

Initializes a new [ReadOnlyStringBuffer](#) with a string.

```
public ReadOnlyStringBuffer(string content)
```

Parameters

content [string](#) 

The string that the buffer will wrap.

ReadOnlyStringBuffer(ReadOnlySpan<char>)

Initializes a new [ReadOnlyStringBuffer](#) by allocating a string from a [ReadOnlySpan<T>](#) .

```
public ReadOnlyStringBuffer(ReadOnlySpan<char> content)
```

Parameters

content [ReadOnlySpan](#)  [<char>](#) 

The span pointing to the string's data.

ReadOnlyStringBuffer(char[])

Initializes a new [ReadOnlyStringBuffer](#) with an array of characters.

```
public ReadOnlyStringBuffer(char[] content)
```

Parameters

content [char](#)[]

The array of characters to use for the buffer.

ReadOnlyStringBuffer(byte[], Encoding?)

Initializes a new [ReadOnlyStringBuffer](#) with an array of bytes and the encoding to use when decoding them.

```
public ReadOnlyStringBuffer(byte[] content, Encoding? encoding = null)
```

Parameters

content [byte](#)[]

The array of bytes to use for the buffer.

encoding [Encoding](#)

The encoding to use when decoding **content**. Use `null` for the [Default](#) encoding.

ReadOnlyStringBuffer(byte*, int, Encoding?)

Initializes a new [ReadOnlyStringBuffer](#) with a pointer referencing an array of bytes and the encoding to use when decoding them.

```
[CLSCompliant(false)]  
public ReadOnlyStringBuffer(byte* contentPtr, int byteCount, Encoding? encoding  
= null)
```

Parameters

contentPtr [byte](#)*

The pointer referecing the array of bytes to use for the buffer.

byteCount [int](#)

The amount of bytes in the array referenced by `contentPtr`.

encoding [Encoding](#)

The encoding to use when decoding `contentPtr`. Use `null` for the [Default](#) encoding.

Remarks

This method is not CLS-compliant due to the unsafe context and the use of pointers.

Property CacheExists

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CacheExists

Whether there's cached data.

```
public bool CacheExists { get; }
```

Property Value

[bool](#)

Property IsEmpty

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsEmpty

Whether the source is completely empty.

```
public bool IsEmpty { get; }
```

Property Value

[bool](#)

Property IsReadOnly

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsReadOnly

Whether the source can be mutated.

```
public bool IsReadOnly { get; }
```

Property Value

[bool](#)

Remarks

Always returns **true**, as [ReadOnlyStringBuffer](#) only supports read-only content (hence the name).

Property Length

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Length

The length of the source.

```
public int Length { get; }
```

Property Value

[int](#)

Property LineCount

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

LineCount

The total amount of lines in the buffer.

```
public int LineCount { get; }
```

Property Value

[int](#)

Remarks

[LineCount](#) automatically builds cache if no cached data exists. No [BuildCache\(\)](#) calls are necessary.

Property this

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

this[int]

Gets a single item out of the buffer.

```
public char? this[int index] { get; }
```

Parameters

index [int](#)

The index of the item to retrieve.

Property Value

[char](#)?

The item.

this[SourceLocation]

Gets a single item out of the buffer.

```
public char? this[SourceLocation location] { get; }
```

Parameters

location [SourceLocation](#)

The location of the item to retrieve.

Property Value

[char](#)?

The item.

this[SourceSpan]

Gets a range of items out of the buffer.

```
public Result<string> this[SourceSpan span] { get; }
```

Parameters

span [SourceSpan](#)

The span of items to get.

Property Value

[Result](#)<[string](#)>

The items.

Method BuildCache

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

BuildCache()

Builds the cache if it doesn't exist yet.

```
public void BuildCache()
```

BuildCache(bool)

Builds the cache. If `forceRebuild` is set to `true`, the cache will be built even if it already exists.

```
public void BuildCache(bool forceRebuild)
```

Parameters

`forceRebuild` [bool](#)

Whether to force the cache to be built even if it exists.

Method CountUntilEndOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilEndOfLine(int, out bool)

Counts the amount of items until the next line separator in the buffer, relative to a given **index**. Only line separators count - regular whitespace do not. CRLF counts as a single line separator, to avoid double counting.

```
public Result<int> CountUntilEndOfLine(int index, out bool isCrLf)
```

Parameters

index [int](#)

The index at which to start counting.

isCrLf [bool](#)

Whether the line separator at which the method stopped is the CR in a CRLF sequence. If true, the next item in the buffer is LF.

Returns

[Result](#)<[int](#)>

The index of the next line separator, relative to an **index**.

Remarks

NOTE

This method allows the EOF index as in-range. The convention is as follows:

- If the index is EOF ([Length](#)), then the output is always 0 and **isCrLf** is always **false**.
- If the index is the last ([Length](#) - 1), then the output is always 0.

i NOTE

`isCrLf` is only `true` if all the following conditions are true:

- The next line start counting from `index` is preceded by a CRLF sequence.
- `index` does not point to the LF in the CRLF sequence.
- `index` does not point to EOF.

Method CountUntilNotWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilNotWhitespace(int)

Counts the amount of items until the next non-whitespace item in the buffer, relative to a given `index`. Line separators are included in the whitespace category.

```
public Result<int> CountUntilNotWhitespace(int index)
```

Parameters

`index` [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next non-whitespace item, relative to a `index`.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that is not whitespace, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that are not whitespace), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method CountUntilWhitespace

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountUntilWhitespace(int)

Counts the amount of items until the next whitespace item in the buffer, relative to a given **index**. Line separators are included in the whitespace category.

```
public Result<int> CountUntilWhitespace(int index)
```

Parameters

index [int](#)

The index at which to start counting.

Returns

[Result](#)<[int](#)>

The index of the next whitespace item, relative to a **index**.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that is whitespace, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that are whitespace), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method CountWhile

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CountWhile(Func<int, char, bool>, int)

Counts the amount of items, starting from a given **index**, while a **predicate** returns **true**.

```
public Result<int> CountWhile(Func<int, char, bool> predicate, int index)
```

Parameters

predicate [Func](#)<[int](#), [char](#), [bool](#)>

A function that takes the current index (relative to **index**), which is incremented every item, and the item associated with it. Execution stops when the predicate returns **false** or the index is out of bounds.

index [int](#)

The index at which to start counting; all indexes will also be given to the **predicate** as an offset that when added to the index of the position equal the actual index.

Returns

[Result](#)<[int](#)>

The amount of items counted.

Remarks

NOTE

This method allows the EOF index as in-range. If the index is EOF ([Length](#)), then the output is always 0.

① NOTE

The return value, when summed with `index`, becomes the index of the first character that fails to verify the `predicate`, counting from `index`. The only exception is if the buffer has been consumed (you pass EOF index or there's no more characters that fail to verify the `predicate`), meaning the return value, when summed with `index` is the value of `Length`, which is also the EOF index.

Method Dispose

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Dispose()

Performs application-defined tasks associated with freeing, releasing, or resetting unmanaged resources.

```
public void Dispose()
```

Method Equals

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Equals(object?)

Determines whether the specified object is equal to the current object.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current object.

Returns

[bool](#)

[true](#) if the specified object is equal to the current object; otherwise, [false](#).

Equals(char[]?)

Checks if an array of characters is equal to the current instance.

```
public bool Equals(char[]? other)
```

Parameters

other [char](#)[]

The array.

Returns

[bool](#)

True if equals.

Equals(IReadOnlyBuffer?)

Checks if another read-only buffer is equal to the current instance.

```
public bool Equals(IReadOnlyBuffer? other)
```

Parameters

other [IReadOnlyBuffer](#)

The other read-only buffer.

Returns

[bool](#)

True if equals.

Equals(ReadOnlyMemory<char>)

Checks if a read-only contiguous region of memory is equal to the current instance.

```
public bool Equals(ReadOnlyMemory<char> other)
```

Parameters

other [ReadOnlyMemory](#) <[char](#)>

The region of memory.

Returns

[bool](#)

True if equals.

Equals(ReadOnlySpan<char>)

Checks if a read-only contiguous region of memory is equal to the current instance.

```
public bool Equals(ReadOnlySpan<char> other)
```

Parameters

other [ReadOnlySpan](#) <[char](#)>

The region of memory.

Returns

[bool](#)

True if equals.

Equals(string?)

Checks if a string is equal to the current instance.

```
public bool Equals(string? other)
```

Parameters

other [string](#)

The string.

Returns

[bool](#)

True if equals.

Method GetHashCode

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetHashCode()

Serves as the default hash function.

```
public override int GetHashCode()
```

Returns

[int](#)

A hash code for the current object.

Method GetLengthOfLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLine(int)

Returns the length of a line given its 0-based line number. Line separators do not count towards the length.

```
public Result<int> GetLengthOfLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result<int>](#)

The length of the line.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLengthOfLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLengthOfLineFromIndex(int)

Returns the length of a line given a 0-based index of one of its items. Line separators do not count towards the length.

```
public Result<int> GetLengthOfLineFromIndex(int index)
```

Parameters

index [int](#)

The 0-based index whose line is considered.

Returns

[Result](#)<[int](#)>

The length of the line.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLine(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
public Result<string> GetLine(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[Result](#)<[string](#)>

The line as an array of items.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLineAsSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpan(int)

Given a 0-based line number, returns the matching line as an array of buffer items.

```
public ReadOnlySpan<char> GetLineAsSpan(int lineNumber)
```

Parameters

lineNumber [int](#)

The 0-based line number.

Returns

[ReadOnlySpan](#) <[char](#)>

The line as a span.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method GetLineAsSpanFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineAsSpanFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public ReadOnlySpan<char> GetLineAsSpanFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[ReadOnlySpan](#) <[char](#)>

The line, as a span.

Method GetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineFromIndex(int)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public Result<string> GetLineFromIndex(int index)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

Returns

[Result](#)<[string](#)>

The line, as an array of items.

Method GetLineNumberFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetLineNumberFromIndex(int)

Determines the 0-based line number of the line that contains the item located at `index`.

```
public Result<int> GetLineNumberFromIndex(int index)
```

Parameters

`index` [int](#)

The index whose parent line's number is to be returned.

Returns

[Result](#)<[int](#)>

The 0-based number of the line that contains item located at `index`.

Remarks

NOTE

This method follows EOF conventions.

Method GetSourceLocation

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceLocation(int)

Converts an index into a location.

```
public Result<SourceLocation> GetSourceLocation(int index)
```

Parameters

index [int](#)

The index.

Returns

[Result](#)<[SourceLocation](#)>

The location.

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered a location.

Method GetSourceSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetSourceSpan(int, int)

Converts the buffer region into a span.

```
public Result<SourceSpan> GetSourceSpan(int startIndex, int endIndex)
```

Parameters

startIndex [int](#) 

The starting index.

endIndex [int](#) 

The end index, which is included in the span.

Returns

[Result](#)<[SourceSpan](#)>

The span.

Method Slice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Slice(int, int)

Slices a region of the buffer.

```
public Result<string> Slice(int start, int length)
```

Parameters

start [int](#)

The index of the first item in the slice.

length [int](#)

The amount of items to slice starting at **start**.

Returns

[Result](#)<[string](#)>

A slice, as an array of items.

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ToString()

Returns the buffer's content as a [string](#) .

```
public override string ToString()
```

Returns

[string](#) 

The buffer's content.

Method TryGetChar

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetChar(int, out char)

Tries to return the item at `index`.

```
public bool TryGetChar(int index, out char item)
```

Parameters

`index` [int](#)

The index of the character.

`item` [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

Remarks

NOTE

This method follows the EOF convention where the EOF character is 0 (`'\0'`) and the return value is `false`, due to EOF not being an actual buffer location.

TryGetChar(SourceLocation, out char)

Tries to return the item at the specified `location`.

```
public bool TryGetChar(SourceLocation location, out char item)
```

Parameters

location [SourceLocation](#)

The item's location.

item [char](#)

The item.

Returns

[bool](#)

False if the buffer is out of bounds or an exception occurred.

Remarks

NOTE

This method follows the EOF convention where the EOF character is 0 (`'\0'`) and the return value is `false`, due to EOF not being an actual buffer location.

Method TryGetLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLine(int, Span<char>)

Given a 0-based line number, assigns the exact line to a result buffer (*destination*). No end of line characters are added.

```
public bool TryGetLine(int lineNumber, Span<char> destination)
```

Parameters

lineNumber [int](#)

The 0-based line number.

destination [Span](#) <[char](#)>

The destination buffer for the line.

Returns

[bool](#)

True if successful.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1).

Method TryGetLineFromIndex

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TryGetLineFromIndex(int, Span<char>)

Tries to read the line that contains the item at `index`. No end of line characters are added.

```
public bool TryGetLineFromIndex(int index, Span<char> destination)
```

Parameters

`index` [int](#)

The index at which to determine what the current line is.

`destination` [Span](#) <[char](#)>

The destination span that will contain the line.

Returns

[bool](#)

True if successful.

Remarks

NOTE

This method follows EOF conventions. EOF is considered a 0-character sequence in line N, where N is [LineCount](#). Keep in mind N does not point to an actual line (it's just a convention), as line numbers are 0-based (meaning the actual last line is located at N - 1). If `index` is EOF, the line will also be EOF.

Method TrySlice

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

TrySlice(int, Span<char>)

Slices a region of the buffer into a performant span.

```
public bool TrySlice(int start, Span<char> slice)
```

Parameters

start [int](#)

The index of the first item in the slice.

slice [Span](#) <[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#)

Remarks

⊗ IMPORTANT

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

TrySlice(SourceSpan, Span<char>)

Slices a region of the buffer into a performant span.

```
public bool TrySlice(SourceSpan sourceSpan, Span<char> slice)
```

Parameters

sourceSpan [SourceSpan](#)

The span indicating what the slice is.

slice [Span](#)<[char](#)>

The span serving as the destination for the slice.

Returns

[bool](#)

Remarks

⊗ **IMPORTANT**

Unlike with other [ReadOnlyStringBuffer](#) methods, this one does not follow EOF conventions and, because of that, does not accept the EOF index (index at [Length](#)), because it is not considered part of any slice.

Operator operator !=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator !=(ReadOnlyStringBuffer, ReadOnlyStringBuffer)

Checks if two read-only string buffers are different.

```
public static bool operator !=(ReadOnlyStringBuffer left, ReadOnlyStringBuffer right)
```

Parameters

left [ReadOnlyStringBuffer](#)

right [ReadOnlyStringBuffer](#)

Returns

[bool](#) 

True if different.

Operator operator ==

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator ==(ReadOnlyStringBuffer, ReadOnlyStringBuffer)

Checks if two read-only string buffers are equals.

```
public static bool operator ==(ReadOnlyStringBuffer left, ReadOnlyStringBuffer right)
```

Parameters

left [ReadOnlyStringBuffer](#)

right [ReadOnlyStringBuffer](#)

Returns

[bool](#) 

True if equals.

Struct SourceLocation

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Specifies the location of an item in a [ISource](#).

```
public readonly struct SourceLocation : IEquatable<SourceLocation>, IEquatable<int>,
    IFormattable, IComparable<SourceLocation>, IComparable<int>
```

Implements

[IEquatable](#)[↗] <[SourceLocation](#)>, [IEquatable](#)[↗] <[int](#)[↗]>, [IFormattable](#)[↗],
[IComparable](#)[↗] <[SourceLocation](#)>, [IComparable](#)[↗] <[int](#)[↗]>

Inherited Members

[object.Equals\(object, object\)](#)[↗] , [object.GetType\(\)](#)[↗] ,
[object.ReferenceEquals\(object, object\)](#)[↗]

Constructors

[SourceLocation\(int, int, int\)](#)

Specifies the location of an item in a [ISource](#).

Properties

[Column](#)

The 0-based column number, which is the index of the item relative to the start of the line it is in.

[Index](#)

The absolute 0-based index of the item in the source.

[Line](#)

The 0-based line number, counting from the start of the source.

Methods

[CompareTo\(SourceLocation\)](#)

Compares the current instance with another object of the same type and returns an integer that indicates whether the current instance precedes, follows, or occurs in the same position in the sort order as the other object.

[CompareTo\(int\)](#)

Compares the index of the location to another index.

[Equals\(SourceLocation\)](#)

Checks if two [SourceLocation](#)s are equal to each other.

[Equals\(int\)](#)

Checks if two indexes are equal to each other.

[Equals\(object?\)](#)

Indicates whether this instance and a specified object are equal.

[GetHashCode\(\)](#)

Returns the hash code for this instance.

[ToString\(\)](#)

Returns a generic string representation of the current instance.

[ToString\(string?, IFormatProvider?\)](#)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

Operators

[operator ==\(SourceLocation, SourceLocation\)](#)

Checks if two [SourceLocation](#)s are equal to each other.

[operator >\(SourceLocation, SourceLocation\)](#)

Checks if `left` is strictly greater than `right`.

[operator >=\(SourceLocation, SourceLocation\)](#)

Checks if `left` is greather than or equal to `right`.

[operator !=\(SourceLocation, SourceLocation\)](#)

Checks if two [SourceLocation](#)s are different from each other.

[operator <\(SourceLocation, SourceLocation\)](#)

Checks if `left` is strictly less than `right`.

[operator <=\(SourceLocation, SourceLocation\)](#)

Checks if `left` is less than or equal to `right`.

Constructor SourceLocation

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

SourceLocation(int, int, int)

Specifies the location of an item in a [ISource](#).

```
public SourceLocation(int index, int line, int column)
```

Parameters

index [int](#)

The 0-based index.

line [int](#)

The 0-based line number.

column [int](#)

The 0-based column number (the index relative to the start of the line).

Property Column

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Column

The 0-based column number, which is the index of the item relative to the start of the line it is in.

```
public int Column { get; }
```

Property Value

[int](#)

Property Index

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Index

The absolute 0-based index of the item in the source.

```
public int Index { get; }
```

Property Value

[int](#)

Property Line

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Line

The 0-based line number, counting from the start of the source.

```
public int Line { get; }
```

Property Value

[int](#)

Method CompareTo

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

CompareTo(int)

Compares the index of the location to another index.

```
public int CompareTo(int other)
```

Parameters

other [int](#)

The index to compare against.

Returns

[int](#)

CompareTo(SourceLocation)

Compares the current instance with another object of the same type and returns an integer that indicates whether the current instance precedes, follows, or occurs in the same position in the sort order as the other object.

```
public int CompareTo(SourceLocation other)
```

Parameters

other [SourceLocation](#)

An object to compare with this instance.

Returns

A value that indicates the relative order of the objects being compared. The return value has these meanings:

Value	Meaning
Less than zero	This instance precedes <i>other</i> in the sort order.
Zero	This instance occurs in the same position in the sort order as <i>other</i> .
Greater than zero	This instance follows <i>other</i> in the sort order.

Method Equals

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current instance.

Returns

[bool](#)

[true](#) if **obj** and this instance are the same type and represent the same value; otherwise, [false](#).

Equals(SourceLocation)

Checks if two [SourceLocation](#)s are equal to each other.

```
public bool Equals(SourceLocation other)
```

Parameters

other [SourceLocation](#)

The other [SourceLocation](#).

Returns

[bool](#)

True if equals.

Equals(int)

Checks if two indexes are equal to each other.

```
public bool Equals(int other)
```

Parameters

other [int](#)

The other location's index.

Returns

[bool](#)

True if equals.

Method GetHashCode

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

[int](#)

A 32-bit signed integer that is the hash code for this instance.

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ToString()

Returns a generic string representation of the current instance.

```
public override string ToString()
```

Returns

[string](#) 

The string representation.

ToString(string?, IFormatProvider?)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

```
public string ToString(string? format, IFormatProvider? formatProvider)
```

Parameters

format [string](#) 

The format. Available formats are: CTOR (constructor-like string) and JSON (struct as JSON).

formatProvider [IFormatProvider](#) 

Unused. Don't bother assigning it anything.

Returns

[string](#) 

The string representation.

Operator operator !=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator !=(SourceLocation, SourceLocation)

Checks if two [SourceLocation](#)s are different from each other.

```
public static bool operator !=(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

True if different.

Operator operator <

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator <(SourceLocation, SourceLocation)

Checks if **left** is strictly less than **right**.

```
public static bool operator <(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

Operator operator <=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator <=(SourceLocation, SourceLocation)

Checks if **left** is less than or equal to **right**.

```
public static bool operator <=(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

Operator operator ==

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator ==(SourceLocation, SourceLocation)

Checks if two [SourceLocation](#)s are equal to each other.

```
public static bool operator ==(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

True if equals.

Operator operator >

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator >(SourceLocation, SourceLocation)

Checks if **left** is strictly greater than **right**.

```
public static bool operator >(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

Operator operator >=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator >=(SourceLocation, SourceLocation)

Checks if **left** is greather than or equal to **right**.

```
public static bool operator >=(SourceLocation left, SourceLocation right)
```

Parameters

left [SourceLocation](#)

right [SourceLocation](#)

Returns

[bool](#)

Struct SourceSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Represents a span taken from a source.

```
public readonly struct SourceSpan : IFormattable
```

Implements

[IFormattable](#)

Inherited Members

[object.Equals\(object, object\)](#) , [object.GetType\(\)](#) ,
[object.ReferenceEquals\(object, object\)](#)

Constructors

[SourceSpan\(SourceLocation, SourceLocation\)](#)

Initializes a new span given a starting and an end indexes.

Fields

[Empty](#)

An empty span.

Properties

[End](#)

The end of the span.

[IsSingleLine](#)

The span is located in a single line.

[Length](#)

The length of the span.

[Start](#)

The start of the span.

Methods

[Equals\(SourceSpan?\)](#)

Checks whether two [SourceSpan](#)s are equals.

[Equals\(object?\)](#)

Indicates whether this instance and a specified object are equal.

[GetHashCode\(\)](#)

Returns the hash code for this instance.

[ToString\(\)](#)

Returns a generic string representation of the current instance.

[ToString\(ISource\)](#)

Given a source, tries to return a string that uses said source as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

[ToString\(string?, IFormatProvider?\)](#)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

Operators

[operator ==\(SourceSpan, SourceSpan\)](#)

Checks whether two [SourceSpan](#)s are equals.

[operator !=\(SourceSpan, SourceSpan\)](#)

Checks whether two [SourceSpan](#)s are different from each other.

Constructor SourceSpan

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

SourceSpan(SourceLocation, SourceLocation)

Initializes a new span given a starting and an end indexes.

```
public SourceSpan(SourceLocation start, SourceLocation end)
```

Parameters

start [SourceLocation](#)

The start index.

end [SourceLocation](#)

The end index.

Exceptions

[ArgumentException](#)

The starting index is greater or equal to the end index.

Field Empty

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

An empty span.

```
public static readonly SourceSpan Empty
```

Returns

[SourceSpan](#)

An empty span.

Property End

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

End

The end of the span.

```
public SourceLocation End { get; }
```

Property Value

[SourceLocation](#)

Property IsSingleLine

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

IsSingleLine

The span is located in a single line.

```
public bool IsSingleLine { get; }
```

Property Value

[bool](#)

Property Length

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Length

The length of the span.

```
public int Length { get; }
```

Property Value

[int](#)

Property Start

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Start

The start of the span.

```
public SourceLocation Start { get; }
```

Property Value

[SourceLocation](#)

Method Equals

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

Equals(object?)

Indicates whether this instance and a specified object are equal.

```
public override bool Equals(object? obj)
```

Parameters

obj [object](#)

The object to compare with the current instance.

Returns

[bool](#)

[true](#) if **obj** and this instance are the same type and represent the same value; otherwise, [false](#).

Equals(SourceSpan?)

Checks whether two [SourceSpan](#)s are equals.

```
public bool Equals(SourceSpan? other)
```

Parameters

other [SourceSpan](#)

The span to check against

Returns

[bool](#) 

True if equals

Method GetHashCode

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

GetHashCode()

Returns the hash code for this instance.

```
public override int GetHashCode()
```

Returns

[int](#)

A 32-bit signed integer that is the hash code for this instance.

Method ToString

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

ToString()

Returns a generic string representation of the current instance.

```
public override string ToString()
```

Returns

[string](#)

The string representation.

ToString(ISource)

Given a source, tries to return a string that uses said source as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

```
public string ToString(ISource source)
```

Parameters

source [ISource](#)

The source.

Returns

[string](#)

The string representation.

ToString(string?, IFormatProvider?)

Given a format, tries to return a string that uses said format as a basis for the representation. If it fails, it defaults to [ToString\(\)](#).

```
public string ToString(string? format, IFormatProvider? formatProvider = null)
```

Parameters

format [string](#)

The format. Available formats are: CTOR (constructor-like string) and JSON (struct as JSON).

formatProvider [IFormatProvider](#)

Unused. Don't bother assigning it anything.

Returns

[string](#)

The string representation.

Operator operator !=

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator !=(SourceSpan, SourceSpan)

Checks whether two [SourceSpan](#)s are different from each other.

```
public static bool operator !=(SourceSpan left, SourceSpan right)
```

Parameters

left [SourceSpan](#)

right [SourceSpan](#)

Returns

[bool](#)

True if different

Operator operator ==

Namespace: [OceanApocalypseStudios.RSML.Sources](#)

Assembly: RSML.dll

operator ==(SourceSpan, SourceSpan)

Checks whether two [SourceSpan](#)s are equals.

```
public static bool operator ==(SourceSpan left, SourceSpan right)
```

Parameters

left [SourceSpan](#)

right [SourceSpan](#)

Returns

[bool](#)

True if equals